Diseño e Implementación de un Algoritmo de Cifrado y Huella Digital con Control de Integridad

Nombre del Autor

Nombre del Segundo Autor

Nombre de la Institución

Nombre del Departamento

Nombre del Curso

Fecha: Abril 2026

Resumen

Este trabajo presenta el diseño e implementación de un algoritmo de cifrado simétrico AES-256-GCM con control de integridad mediante huella digital SHA-256, desarrollado en Python. El sistema proporciona confidencialidad mediante cifrado autenticado, integridad de datos mediante verificación de etiquetas de autenticación GCM y huellas digitales SHA-256, y una interfaz de línea de comandos para facilitar su uso. La implementación emplea derivación de claves PBKDF2-SHA256 con 480,000 iteraciones (recomendación OWASP 2023), sales aleatorias de 16 bytes y vectores de inicialización únicos de 12 bytes por operación. Se incluyen pruebas unitarias e integración con 59 casos de prueba alcanzando 89% de cobertura de código. El proyecto demuestra la aplicabilidad práctica de los conceptos de cifrado simétrico, autenticación de datos y criptografía en escenarios reales.

Palabras clave: cifrado AES-GCM, huella digital SHA-256, PBKDF2, integridad de datos, criptografía simétrica, Python

1. Introducción

En el ámbito de la seguridad informática, la protección de la confidencialidad e integridad de los datos es fundamental. El cifrado permite proteger la confidencialidad de la información, mientras que las huellas digitales (funciones hash) permiten verificar su integridad y detectar alteraciones. En escenarios reales, estos mecanismos se combinan para proteger archivos, mensajes y evidencias digitales en sistemas y redes (Stallings, 2017).

El presente trabajo desarrolla un algoritmo que integra cifrado simétrico AES-256 en modo GCM (Galois/Counter Mode) con huellas digitales SHA-256 para control de integridad. Se optó por mejorar un esquema de cifrado existente, agregando componentes de seguridad como sales aleatorias, vectores de inicialización únicos, derivación de claves resistente a ataques de fuerza bruta y verificación de integridad dual (etiqueta de autenticación GCM más huella digital SHA-256). La implementación se realizó en Python utilizando la biblioteca criptográfica `cryptography` (pyca), un estándar de la industria.

Los objetivos específicos del proyecto son: (a) implementar cifrado/descifrado AES-256-GCM para cadenas de texto y archivos pequeños, (b) incorporar verificación de integridad mediante huella digital SHA-256, (c) desarrollar una interfaz de línea de comandos funcional, (d) validar el funcionamiento con pruebas unitarias e integración, y (e) documentar el algoritmo y sus decisiones de diseño en este informe.

2. Descripción del Algoritmo

2.1 Arquitectura General

El sistema se organiza en cinco módulos principales que siguen principios de responsabilidad única y modularidad:

Módulo	Responsabilidad
cipher/	Cifrado y descifrado AES-256-GCM
key_utils/	Derivación de claves PBKDF2-SHA256 y generación de sales
hash_utils/	Cómputo y verificación de huellas digitales SHA-256
validators/	Validación de entradas y manejo de archivos
cli/	Interfaz de línea de comandos (encrypt, decrypt, hash, verify)

2.2 Flujo de Cifrado

El proceso de cifrado sigue los siguientes pasos, ejecutados secuencialmente:

1. El usuario proporciona una contraseña y el texto o archivo a cifrar.
2. Se genera una sal aleatoria de 16 bytes mediante os.urandom().
3. Se deriva una clave de 32 bytes (256 bits) usando PBKDF2-SHA256 con la sal y 480,000 iteraciones.
4. Se genera un vector de inicialización (IV/nonce) aleatorio de 12 bytes.
5. Se cifra el texto plano con AES-256-GCM, obteniendo el ciphertext y una etiqueta de autenticación de 16 bytes.
6. Se calcula la huella digital SHA-256 del texto plano original.
7. Se empaqueta el resultado en formato binario: [sal 16B][IV 12B][longitud_metadata 4B][metadata JSON][ciphertext+tag].

2.3 Flujo de Descifrado

El proceso de descifrado realiza:

1. Se lee el archivo cifrado y se extraen los componentes: sal, IV, metadata y ciphertext+tag.
2. Se deriva la clave usando la contraseña proporcionada y la sal almacenada.
3. Se descifra con AES-256-GCM verificando la etiqueta de autenticación.
4. Si la etiqueta no coincide, se lanza IntegrityError indicando datos alterados.
5. Si la contraseña es incorrecta, se lanza InvalidKeyError.
6. Si la verificación es exitosa, se devuelve el texto plano original.

2.4 Huella Digital SHA-256

La huella digital (fingerprint) SHA-256 se utiliza para verificar la integridad de los datos independientemente del cifrado. Se implementa un mecanismo dual de integridad: por un lado, la etiqueta de autenticación GCM asegura que el ciphertext no haya sido modificado; por otro lado, la huella SHA-256 permite verificar que el texto descifrado coincide con el original. El módulo `hash_utils/sha256.py` provee las funciones `compute_sha256()` para generar la huella y `verify_fingerprint()` para compararla, retornando un `IntegrityReport` con el resultado de la verificación.

3. Técnica Usada y Justificación

3.1 Cifrado AES-256-GCM

Se seleccionó AES-256 en modo GCM (Galois/Counter Mode) como algoritmo de cifrado por las siguientes razones: (a) proporciona cifrado autenticado (AEAD), garantizando tanto

confidencialidad como integridad en una sola operación; (b) es el estándar adoptado por NIST y ampliamente implementado en sistemas reales (NIST, 2007); (c) elimina la necesidad de relleno (padding) que es vulnerable a ataques oracle; y (d) permite detectar modificaciones en el ciphertext mediante la etiqueta de autenticación de 128 bits. Se descartó el modo CBC porque no incluye autenticación intrínseca, requiriendo un mecanismo HMAC adicional (Dworkin, 2001).

3.2 Derivación de Claves con PBKDF2-SHA256

Para convertir contraseñas de usuario en claves criptográficas seguras, se empleó PBKDF2-HMAC-SHA256 con 480,000 iteraciones, cumpliendo la recomendación de OWASP (2023). Este número de iteraciones proporciona un balance adecuado entre seguridad y rendimiento en hardware moderno. Cada operación de derivación utiliza una sal aleatoria única de 16 bytes, lo que previene ataques de tablas rainbow y garantiza que la misma contraseña produce claves diferentes en cada ejecución (Kaliski, 2000).

3.3 Huella Digital SHA-256

SHA-256 se seleccionó como función hash para la huella digital por ser parte de la familia SHA-2, aprobada por NIST (2012) y resistente a los ataques conocidos de colisión y preimagen. A diferencia de SHA-1 (vulnerable a ataques de colisión) y MD5 (criptográficamente roto), SHA-256 ofrece un nivel de seguridad de 128 bits contra ataques de colisión, adecuado para verificación de integridad. La implementación utiliza la biblioteca `cryptography` que delega las operaciones criptográficas a OpenSSL, asegurando rendimiento y seguridad auditada.

3.4 Formato de Archivo Cifrado

Se diseñó un formato binario autocontenido para los archivos cifrados que almacena todos los parámetros necesarios para el descifrado. La estructura es: [sal 16 bytes][IV 12

bytes][longitud_metadata 4 bytes big-endian][metadata JSON][ciphertext+tag]. La metadata incluye el nombre del archivo original, su tamaño y los parámetros de derivación de clave. Este diseño permite que un solo archivo contenga toda la información necesaria para la recuperación, sin requerir parámetros externos.

4. Ejemplos de Entrada/Salida y Resultados de Pruebas

4.1 Cifrado y Descifrado de Texto

Ejemplo de cifrado de texto con el comando CLI:

```
$ python -m algoritmo_cifrado.cli.main encrypt -i "Hola Mundo" -p mi_password
a1b2c3d4e5f6...  (hex output)

$ python -m algoritmo_cifrado.cli.main decrypt -i archivo.enc -p mi_password
-o salida.txt
Decrypted to salida.txt
```

4.2 Huella Digital y Verificación de Integridad

Cómputo de huella digital SHA-256:

```
$ python -m algoritmo_cifrado.cli.main hash -i "Hola Mundo"
2f3c4d5e6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3

$ python -m algoritmo_cifrado.cli.main verify -i "Hola Mundo" -f
2f3c4d5e6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3
Valid: Fingerprint matches
```

4.3 Resultados de Pruebas

Se implementaron 59 pruebas automáticas cubriendo los siguientes escenarios:

Categoría	Pruebas	Resultado
Cifrado AES-GCM	9	9 pasadas
Comandos CLI	6	6 pasadas
CLI Main	10	10 pasadas
Hash SHA-256	7	7 pasadas
Derivación de claves	7	7 pasadas
Validadores	20	20 pasadas

Cobertura total del código: 89%, superando el umbral mínimo del 80% establecido. Las pruebas incluyen casos normales (cifrado/descifrado round-trip, verificación de huella), casos de error (clave incorrecta, datos alterados, entrada vacía) y casos de integración (flujo completo CLI, archivos).

4.4 Detección de Alteraciones

Se verificó que el sistema detecta correctamente datos alterados: al modificar un solo byte del ciphertext, la verificación de la etiqueta de autenticación GCM falla y se lanza una excepción `IntegrityError`. De igual forma, al usar una contraseña incorrecta, el descifrado falla con `InvalidKeyError`. Estos mecanismos garantizan que cualquier modificación no autorizada del ciphertext es detectada.

5. Conclusiones

Se implementó exitosamente un algoritmo de cifrado simétrico AES-256-GCM con control de integridad mediante huella digital SHA-256, cumpliendo con los objetivos establecidos. El

sistema proporciona confidencialidad e integridad de datos, con una arquitectura modular que facilita su comprensión, mantenimiento y extensión.

5.1 Alcances

El sistema implementa cifrado y descifrado de cadenas de texto y archivos pequeños (<1MB), verificación de integridad dual (GCM tag + SHA-256), derivación segura de claves con PBKDF2-SHA256 (480K iteraciones), interfaz de línea de comandos completa, y un conjunto de 59 pruebas con 89% de cobertura. La implementación utiliza exclusivamente generadores de números aleatorios criptográficamente seguros (os.urandom) y evita la filtración de información en mensajes de error.

5.2 Limitaciones

Las limitaciones identificadas son: (a) no soporta archivos mayores a 1MB de manera eficiente, ya que carga completamente el contenido en memoria; (b) el formato de archivo cifrado es propietario y no compatible con otras herramientas; (c) no implementa gestión de contraseñas ni almacenamiento seguro de claves; (d) el modo GCM requiere un nonce único por cada operación de cifrado con la misma clave, lo cual no se garantiza automáticamente en todos los escenarios de uso; y (e) la contraseña se transmite como argumento de línea de comandos, lo que puede exponerla en el historial del shell.

6. Referencias

Dworkin, M. (2001). Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM). NIST Special Publication 800-38D. National Institute of Standards and Technology.

Kaliski, B. (2000). PKCS #5: Password-Based Cryptography Specification Version 2.0. RFC 2898. Internet Engineering Task Force.

NIST. (2007). Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST Special Publication 800-38D.

NIST. (2012). Secure Hash Standard (SHS). FIPS PUB 180-4. National Institute of Standards and Technology.

OWASP. (2023). Password Storage Cheat Sheet. OWASP Foundation. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Stallings, W. (2017). Cryptography and Network Security: Principles and Practice (7th ed.). Pearson Education.

The Python Cryptographic Authority. (2024). cryptography documentation. https://cryptography.io/

Apéndice A: Enlace al Repositorio

El código fuente del proyecto, las pruebas y este informe están disponibles en el repositorio de GitHub:

[ENLACE_DEL_REPOSITORIO_AQUI]

El repositorio incluye: código fuente completo, suite de pruebas (59 tests con 89% de cobertura), README con instrucciones de uso, y evidencia de ejecución de las pruebas.